

AGENTS.md

AGENTS.md

| Repository-wide instructions for AI coding agents and human contributors.

1. Project Mission

This project implements an **AI-assisted insurance benefit assessment system**.

The system accepts:

- insurance contracts and policy documents;
- insurance certificates;
- diagnosis certificates;
- surgery certificates;
- hospitalization/discharge certificates;
- medical expense documents;
- accident-related documents;
- other insurance claim evidence.

The system extracts structured facts from those documents, resolves the applicable insurance policy version, identifies candidate coverages, evaluates benefit conditions, and calculates an **estimated insurance benefit**.

The MVP focuses on **fixed-benefit third-sector insurance**, including:

- cancer diagnosis benefits;
- specified disease diagnosis benefits;
- acute myocardial infarction diagnosis benefits;
- disease surgery benefits;
- injury surgery benefits;
- fracture diagnosis benefits;

- hospitalization daily benefits.

The following are later-phase features unless explicitly assigned:

- full indemnity medical insurance calculation;
 - automobile insurance;
 - liability insurance;
 - workers compensation insurance;
 - complex disability assessment;
 - fully automated claim submission.
-

2. Source of Truth

The project documents define the intended system.

Use the following precedence when requirements conflict:

1. Requirements Definition / SRS
2. System Architecture
3. ERD
4. System Functional Structure
5. User Flow
6. UI Specification
7. Existing implementation
8. Comments and temporary notes

Existing code is **not automatically the source of truth** if it conflicts with the approved architecture.

If an implementation request conflicts with a higher-level design document:

- do not silently reinterpret the requirement;
- do not change an insurance rule based on assumption;
- clearly identify the conflict;
- implement only what can be implemented safely.

Never invent missing insurance rules.

3. Non-Negotiable Architecture Rules

CRITICAL-01 — LLMs must never directly determine an insurance benefit amount

Never let an LLM directly determine an insurance benefit amount.

Forbidden architecture:

```
Diagnosis document
→ LLM
→ "Benefit = 30,000,000 KRW"
→ User
```

Required architecture:

```
Document
→ OCR
→ AI Fact Extraction
→ Verified Facts
→ Policy Version Resolver
→ Coverage Matcher
→ Eligibility Engine
→ Calculation Engine
→ Evidence
→ Result
```

AI may help understand documents.

AI must not be the authoritative calculator.

CRITICAL-02 — Only deterministic logic produces monetary results

Insurance benefit calculation must use:

```
Verified Facts
+
Contract Coverage
+
Policy Version
+
```

```
Rule Version
=
Coverage Assessment
+
Benefit Calculation
+
Evidence
```

This is the primary system invariant.

Same verified inputs plus the same versioned rules must produce the same result.

```
Same Input
+
Same Rule Version
=
Same Result
```

CRITICAL-03 — Every material calculation value needs provenance

A material value used in insurance assessment must be traceable to at least one approved source.

Examples:

```
diagnosis.code
→ VerifiedFact
→ MedicalDocument
→ Page
→ Bounding Box
```

```
insured_amount
→ ContractCoverage
→ InsuranceContract
```

```
payment_rate
→ RuleVersion
→ PolicyClause
→ PolicyVersion
```

Do not use unexplained values in calculations.

CRITICAL-04 — OCR output is not a verified fact

Never treat:

OCRResult

as equivalent to:

VerifiedFact

Required flow:

```
MedicalDocument  
→ OCRResult  
→ ExtractedFact  
→ VerifiedFact  
→ Assessment
```

Insurance-critical fields must satisfy the configured verification policy before calculation.

CRITICAL-05 — Policy version must be resolved

A calculation must not silently use:

- the newest policy;
- the newest rule;
- a similar product;
- a generic insurance rule.

The applicable version must be resolved from relevant contract and event data.

If the applicable version cannot be determined:

MANUAL_REVIEW

or an equivalent explicit unresolved state must be returned.

CRITICAL-06 — Historical calculations are immutable records

Never silently recalculate historical claims with newly deployed rules.

The following are versioned concepts:

```
ProductVersion
PolicyVersion
RuleVersion
CalculationVersion
```

A recalculation creates a new calculation version.

Example:

```
Calculation V1
is_current = false

Calculation V2
is_current = true
```

Do not destroy V1.

4. Domain Model

The primary domain model is:

```
User
|
├─ Insured
├─ InsuranceContract
|   └─ ContractCoverage
└─ Claim
    ├─ Accident
    ├─ MedicalDocument
    |   └─ OCRResult
    |       └─ ExtractedFact
    |           └─ VerifiedFact
    └─ CoverageAssessment
```

```

├─ AssessmentRuleResult
├─ BenefitCalculation
├─ Evidence
├─ Review

```

Insurance knowledge is modeled separately:

```

InsuranceCompany
↓
InsuranceProduct
↓
ProductVersion
↓
Policy
↓
PolicyVersion
↓
PolicyClause
↓
Coverage
↓
BenefitRule
↓
RuleVersion

```

Do not collapse these concepts for convenience.

5. Repository Structure

Use clear application and domain boundaries.

Preferred logical structure:

```

/
├─ apps/
│   ├── web/
│   ├── admin/
│   ├── api/
│   └─ worker/
├─ domain/
│   ├── auth/
│   ├── user/
│   ├── contract/
│   └─ claim/

```

```

|   |   | document/
|   |   | fact/
|   |   | policy/
|   |   | rule/
|   |   | assessment/
|   |   | calculation/
|   |   | evidence/
|   |   | review/
|   |   | audit/
|   |
|   |--- infrastructure/
|   |   | database/
|   |   | storage/
|   |   | queue/
|   |   | ai/
|   |   | ocr/
|   |   | search/
|   |   | security/
|   |
|   |--- prompts/
|   |
|   |--- shared/
|   |   | types/
|   |   | errors/
|   |   | security/
|   |   | utils/
|   |
|   |--- migrations/
|   |
|   |--- tests/
|   |   | unit/
|   |   | integration/
|   |   | rules/
|   |   | security/
|   |   | e2e/
|   |
|   |--- docs/
|   |
|   |--- scripts/
|   |
|   |--- AGENTS.md

```

Adapt folder syntax to the chosen framework, but preserve the domain boundaries.

Do not introduce microservices merely because logical services exist in the architecture.

The MVP preference is:

```

Modular Monolith
+

```

6. Domain Ownership

Each domain owns its data and business logic.

Domain	Primary entities
Auth	User, Consent
Contract	Insured, InsuranceContract, ContractCoverage
Policy	InsuranceCompany, InsuranceProduct, ProductVersion, Policy, PolicyVersion, PolicyClause, Coverage
Rule	BenefitRule, RuleVersion, RuleCondition
Claim	Claim, Accident
Document	MedicalDocument
OCR	OCRResult
Fact	ExtractedFact, VerifiedFact, Diagnosis, Surgery, Hospitalization
Assessment	CoverageAssessment, AssessmentRuleResult
Calculation	BenefitCalculation
Evidence	Evidence
Review	Review, ReviewAssignment
Audit	AuditLog

A domain must not casually modify another domain's persistence model directly.

Prefer domain APIs/services or explicit application-level orchestration.

7. Claim Lifecycle

Claim is the central aggregate for an insurance assessment.

Supported primary states:

```
DRAFT
DOCUMENT_REQUIRED
DOCUMENT_PROCESSING
USER_VERIFICATION
```

ASSESSING
MANUAL_REVIEW
COMPLETED
FAILED
CANCELLED
CLOSED

Valid high-level flow:

DRAFT
→ DOCUMENT_REQUIRED
→ DOCUMENT_PROCESSING
→ USER_VERIFICATION
→ ASSESSING

ASSESSING may transition to:

COMPLETED
MANUAL_REVIEW
DOCUMENT_REQUIRED
FAILED

MANUAL_REVIEW may transition to:

ASSESSING
DOCUMENT_REQUIRED
COMPLETED

Do not create arbitrary string states inside controllers.

State transitions must be:

- centralized;
- testable;
- validated;
- auditable when material.

8. Document / OCR Rules

Document processing follows:

```
Upload
→ File validation
→ Malware scan
→ Secure storage
→ OCR job
→ OCRResult
→ AI extraction
→ ExtractedFact
```

Validate at minimum:

- allowed MIME type;
- file signature/magic bytes;
- configured file size;
- upload authorization;
- malware scan status.

Supported MVP formats may include:

```
JPG
JPEG
PNG
HEIC
PDF
```

Never rely only on the filename extension.

Do not expose private documents through permanent public URLs.

9. AI Rules

AI is allowed to:

- classify documents;
- extract diagnosis names;
- extract diagnosis codes;
- extract diagnosis dates;
- extract surgery names;
- extract surgery dates;

- extract hospitalization dates;
- identify possible policy clauses;
- assist coverage candidate retrieval;
- normalize natural-language information;
- generate user-facing explanations from approved structured results.

AI must not independently create authoritative values for:

- insured amounts;
 - payment rates;
 - deductibles;
 - waiting periods;
 - exclusion periods;
 - maximum benefit limits;
 - disability percentages;
 - final insurance benefit amounts.
-

Structured AI output is mandatory

Do not pass uncontrolled free-form LLM text directly into business logic.

Expected pattern:

```
{
  "facts": [
    {
      "factType": "DIAGNOSIS_CODE",
      "value": "I21.9",
      "confidence": 0.96,
      "page": 1
    }
  ]
}
```

Validate AI output against a schema.

On invalid output:

```
retry
or
manual review
```

Do not guess missing fields.

10. Policy Version Rules

Before material insurance assessment, resolve:

```
product_version_id
policy_version_id
rule_version_id
```

Inputs may include:

```
productVersion
contractDate
coverageStartDate
coverageEndDate
accidentDate
diagnosisDate
```

Expected resolver behavior:

```
Exactly 1 applicable version
→ resolved
```

```
0 applicable versions
→ unresolved
→ MANUAL_REVIEW
```

```
Multiple conflicting versions
→ VERSION_CONFLICT
→ MANUAL_REVIEW
```

Never choose the latest version merely because resolution failed.

11. Coverage Matching Rules

Coverage matching answers:

| Which subscribed coverages should be evaluated for this claim?

It does **not** answer:

| Will this benefit be paid?

Invariant:

```
Coverage Match
≠
PAYABLE
```

Input may include:

```
VerifiedFacts
ContractCoverages
Coverage aliases
Structured policy metadata
```

Output must be candidate coverages.

12. Rule Engine Rules

MVP rules should be declarative.

Preferred representation:

```
conditions:
  all:
    - fact: diagnosis.code
      operator: IN
      value: AMI_CODES

    - fact: diagnosis.date
```

```
operator: GTE
valueFrom: coverage.startDate
```

Allowed operators should be explicitly whitelisted.

Example:

```
EQ
NE
GT
GTE
LT
LTE
IN
NOT_IN
BETWEEN
EXISTS
NOT_EXISTS
```

Do not allow arbitrary executable code in a rule.

13. Eligibility Rules

Evaluation should preserve semantic stages.

Typical order:

```
Eligibility
→ Exclusion
→ Reduction
→ Limit
→ Calculation eligibility
```

Standard results:

```
PAYABLE
LIKELY_PAYABLE
ADDITIONAL_INFO_REQUIRED
MANUAL_REVIEW
LIKELY_NOT_PAYABLE
POSSIBLE_EXCLUSION
NOT_PAYABLE
UNDETERMINED
```

Do not convert every unresolved result into `NOT_PAYABLE`.

`NOT_PAYABLE` and `UNDETERMINED` have different meanings.

14. Calculation Rules

Only deterministic application/rule logic may create a final monetary calculation.

Example:

```
insuredAmount = 30,000,000
paymentRate   = 1.0

benefitAmount =
insuredAmount × paymentRate
```

Do not calculate material insurance amounts in the frontend.

Do not ask an LLM to perform the authoritative calculation.

Money types

Do not use binary floating-point types for monetary values.

Avoid:

```
float
double
```

Prefer:

```
integer KRW
```

or an appropriate:

```
Decimal
```

type.

Rounding rules must be explicit when required.

Calculation snapshot

A `BenefitCalculation` must retain sufficient data to reproduce the result.

At minimum capture:

```
contractCoverage
insuredAmount
verifiedFacts
policyVersion
ruleVersion
calculationRule
paymentRate or equivalent parameters
formula
input snapshot
final result
calculatedAt
```

Never store only the final amount.

15. Evidence Rules

Every material assessment must be explainable.

Preferred chain:

```
BenefitCalculation
→ RuleVersion
→ PolicyClause
→ PolicyVersion
```

and:

```
BenefitCalculation
→ VerifiedFact
→ MedicalDocument
→ Page
→ Bounding Box
```

A user-facing result should be able to answer:

- Which contract was evaluated?
- Which coverage was evaluated?

- Which policy version was applied?
- Which clause supported the rule?
- Which facts were used?
- Which document contained those facts?
- Which calculation formula was applied?

Evidence is a core business feature, not optional diagnostic metadata.

16. Human Review Rules

Prefer manual review instead of fabricated certainty when:

- a critical OCR field has insufficient confidence;
- diagnosis name and code materially conflict;
- policy version cannot be resolved;
- multiple versions conflict;
- no applicable rule exists;
- applicable rules conflict;
- a material exclusion is ambiguous;
- medical causation requires professional judgment;
- disability assessment requires professional judgment;
- mandatory evidence is missing;
- a high-risk result cannot be justified from structured evidence.

Manual review must preserve:

```
previous value/result  
new value/result  
reviewer  
timestamp  
reason
```

17. Database Rules

Respect the ERD.

Do not merge the following concepts:

InsuranceProduct
≠
InsuranceContract

Coverage
≠
ContractCoverage

Policy
≠
PolicyVersion

BenefitRule
≠
RuleVersion

OCRResult
≠
ExtractedFact
≠
VerifiedFact

CoverageAssessment
≠
BenefitCalculation

Use migrations for schema changes.

Do not manually patch production schema as an undocumented development shortcut.

Primary keys

Prefer UUIDs for internal entity identifiers.

Example:

```
claim_id = UUID
```

Human-facing display identifiers may be separate:

```
CLM-20260818-000001
```

Dates

Business dates should generally be modeled as dates:

```
contract_date  
accident_date  
diagnosis_date  
coverage_start_date
```

Use timezone-aware timestamps for system events:

```
created_at  
updated_at  
approved_at  
calculated_at
```

JSON usage

JSON is appropriate for:

- raw OCR output;
- raw AI output;
- bounding boxes;
- versioned rule definitions;
- calculation input snapshots;
- execution traces.

Do not place the entire relational insurance domain into a single JSON column.

18. API Rules

APIs must use explicit authorization and validation.

Example:

```
GET /claims/{claimId}
```

Authentication alone is insufficient.

The service must verify:

| Does the authenticated actor have permission to access this specific claim?

Apply object-level authorization to:

- claims;
- insurance contracts;
- medical documents;
- calculations;
- evidence;
- reviews.

API errors

Use explicit domain errors.

Examples:

```
POLICY_VERSION_NOT_FOUND  
POLICY_VERSION_CONFLICT  
RULE_NOT_FOUND  
RULE_CONFLICT  
UNVERIFIED_FACT  
REQUIRED_DOCUMENT_MISSING  
CALCULATION_FAILED  
ACCESS_DENIED
```

Do not convert a failed calculation into a successful 0 KRW result.

19. Frontend Rules

Frontend responsibilities:

- display state;
- collect user input;
- perform basic presentation validation;
- show OCR-extracted information;
- allow fact verification;
- display assessments;
- display calculation results;
- display evidence.

Frontend must not become the source of insurance business logic.

Confidence wording

Keep these concepts separate:

OCR confidence

and:

benefit assessment result

Do not display:

OCR confidence 96%

as:

96% probability of insurance payment

unless a separately approved model and definition explicitly exists.

Result presentation

A calculated benefit should expose:

Coverage name
Assessment status
Estimated benefit
Summary reason
Calculation details
Evidence

Every monetary result should provide an affordance equivalent to:

View calculation basis

20. Security Rules

Security is a functional requirement.

Always consider:

- authentication;
- authorization;
- least privilege;
- sensitive data exposure;
- file upload attack surface;
- injection;
- secret management;
- logging;
- auditability.

Object authorization

Never implement resource access based only on a valid token.

Example:

```
GET /claims/{claimId}
```

must validate access to that exact `claimId`.

This applies equally to:

```
contracts
documents
calculations
evidence
reviews
```

File security

Private medical and insurance documents must not use permanent public URLs.

Preferred flow:

```
Authenticated request
→ authorization
→ short-lived signed access
→ encrypted object storage
```

21. Privacy Rules

This system may process highly sensitive information.

Examples:

- diagnoses;
- medical records;
- medical treatment history;
- insurance contracts;
- personal identifiers.

Follow data minimization.

Do not duplicate sensitive personal data across unrelated tables.

Do not send unnecessary personal information to external OCR/AI providers.

Where practical:

```
full document
→ relevant pages
→ minimum necessary content
→ external AI
```

Keep provider integrations behind a dedicated gateway/adaptor.

22. Audit Rules

Audit material actions.

At minimum consider:

```
LOGIN
LOGIN_FAIL

DOCUMENT_UPLOAD
DOCUMENT_VIEW
DOCUMENT_DOWNLOAD

OCR_EXECUTE
FACT_MODIFY

CONTRACT_CREATE
CONTRACT_MODIFY

POLICY_CREATE
POLICY_VERSION_CREATE

RULE_CREATE
RULE_MODIFY
RULE_TEST
RULE_APPROVE
RULE_DEPLOY

CALCULATION_EXECUTE
CALCULATION_RECALCULATE

REVIEW_OVERRIDE

PERMISSION_CHANGE
```

Audit records should make it possible to reconstruct material changes.

23. Async Job Rules

Long-running tasks should not block normal HTTP request handling.

Preferred async tasks:

- OCR;
- AI document extraction;
- policy document ingestion;
- embeddings/indexing;
- bulk assessment;
- notifications.

Standard job states:

```
QUEUED  
PROCESSING  
SUCCEEDED  
FAILED  
RETRYING  
CANCELLED
```

Retries must be bounded.

Never implement infinite retry loops.

24. Error Handling

Do not swallow business errors.

Forbidden:

```
catch Exception:  
    return null
```

Do not treat `null`, `0`, or an empty array as a universal failure state.

Explicitly distinguish:

```
No applicable coverage
```

from:

```
System failed to determine coverage
```

and:

```
Benefit amount is legitimately zero
```

from:

```
Calculation failed
```

Fail safely.

25. Testing Strategy

Every material feature should be evaluated at the appropriate level:

```
Unit  
Integration  
Rule  
Regression  
Security  
E2E
```

Highest-priority unit-test targets:

1. Policy Version Resolver
 2. Rule Parser
 3. Rule Validator
 4. Eligibility Engine
 5. Calculation Engine
 6. Coverage Matcher
 7. Fact verification logic
 8. Claim State Machine
-

26. Rule Regression Testing

Every material rule must have positive and negative cases.

Example golden case:

CASE: AMI-001

Contract coverage:

Acute myocardial infarction diagnosis benefit

Insured amount:

30,000,000 KRW

Diagnosis:

I21.9

Coverage started:

Before diagnosis

Exclusion:

None

Expected:

PAYABLE

Expected amount:

30,000,000 KRW

Negative cases should include, where applicable:

- event before coverage start;
- code not covered;
- exclusion applies;
- waiting period not satisfied;
- required document missing;
- policy version unresolved;
- rule missing;
- unverified critical fact.

Changing a rule requires running existing regression cases.

Do not update expected test results merely to make a changed rule pass unless the rule change intentionally changes the expected business outcome.

27. Coding Conventions

Prefer:

- clear domain names;
- small functions;
- explicit types;
- explicit domain errors;
- deterministic business logic;
- dependency injection for external providers;
- testable pure functions for insurance calculations.

Prefer:

```
insuranceContract  
contractCoverage  
policyVersion  
verifiedFact  
coverageAssessment  
benefitCalculation
```

Avoid meaningless domain abbreviations such as:

```
ic  
cv  
pv  
vf  
bc
```

unless they are strictly local and unambiguous.

Insurance constants

Do not hardcode product-specific insurance terms into generic application logic.

Avoid:

```
CANCER_PAYMENT_RATE = 1.0
```

if that rate belongs to a particular policy/rider/version.

Store the authoritative value in versioned policy/rule data.

28. Change Management

Before changing code, determine:

1. Which domain owns the behavior?
2. Which requirement is affected?
3. Which entities are affected?
4. Does this alter a policy/rule?
5. Can historical calculations change?
6. Does a migration need to be created?
7. Is security affected?
8. Is privacy affected?
9. Which tests must change?
10. Is recalculation required?

Keep changes scoped to the requested task.

Do not perform unrelated large-scale refactoring without a clear technical necessity.

29. AI Agent Working Procedure

For each coding task:

Step 1 — Inspect

Before creating new code, locate:

- existing module;
- related domain entity;
- related service;
- existing API;

- tests;
- migrations;
- current rules;
- current error types.

Do not duplicate existing functionality.

Step 2 — Identify domain ownership

State internally which domain owns the requested behavior.

Examples:

```
Policy version resolution  
→ Policy / Assessment domain
```

```
Insurance amount arithmetic  
→ Calculation domain
```

Step 3 — Identify invariants

Check whether the requested change touches any critical invariant.

Especially:

```
AI must not calculate benefits directly.
```

```
Only verified facts enter authoritative calculations.
```

```
Policy and rule versions must be explicit.
```

```
Historical results must remain reproducible.
```

Step 4 — Implement the minimum coherent change

Modify only what is necessary to complete the assigned functionality.

Avoid speculative architecture.

Step 5 — Add tests

At minimum test:

- normal case;
- critical invalid input;
- relevant domain error;
- authorization if applicable.

Insurance rule changes require rule regression tests.

Step 6 — Run validation

Run the relevant project commands for:

- formatting;
- linting;
- type checking;
- unit tests;
- integration tests;
- migrations where relevant.

Do not claim success if a required test fails.

Step 7 — Review sensitive-data impact

Check whether the implementation:

- logs medical data;
- exposes document URLs;
- sends unnecessary PII to AI;
- weakens authorization;

- changes audit behavior.
-

Step 8 — Report

Summarize exactly what changed and what remains unresolved.

30. Definition of Done

A feature is done only when all applicable conditions are met.

- Requirement implemented.
 - Code builds.
 - Types/checks pass.
 - Relevant tests pass.
 - Domain boundaries preserved.
 - Domain errors handled.
 - Authorization checked.
 - Security implications reviewed.
 - Sensitive-data logging reviewed.
 - Database migration included if required.
 - Audit event added if required.
 - Rule regression test added for rule changes.
 - Historical calculation reproducibility preserved.
 - Evidence generation preserved for material calculations.
 - Documentation updated when architecture or public behavior changed.
-

31. Implementation Order

Unless a task explicitly overrides it, prioritize the MVP in this order.

Phase 1 — Foundation

1. Repository structure

2. Configuration
 3. Database
 4. Migration framework
 5. Authentication
 6. RBAC
 7. Audit foundation
-

Phase 2 — Insurance Master

1. InsuranceCompany
 2. InsuranceProduct
 3. ProductVersion
 4. Policy
 5. PolicyVersion
 6. PolicyClause
 7. Coverage
 8. basic Rule model
-

Phase 3 — Contract

1. Insured
 2. InsuranceContract
 3. ContractCoverage
 4. contract APIs
 5. contract UI
-

Phase 4 — Claim

1. Claim
2. Accident
3. Claim State Machine

4. claim APIs/UI
-

Phase 5 — Document

1. document upload
 2. secure storage
 3. file validation
 4. malware scanning integration
 5. document metadata
-

Phase 6 — OCR / AI

1. OCR provider abstraction
 2. OCR worker
 3. OCRResult
 4. AI extraction provider abstraction
 5. ExtractedFact
 6. VerifiedFact
 7. fact verification UI
-

Phase 7 — Insurance Decision

1. Policy Version Resolver
 2. Coverage Matcher
 3. Rule parser
 4. Rule validator
 5. Rule executor
 6. Eligibility Engine
-

Phase 8 — Calculation

1. Calculation Engine

2. Calculation snapshot
 3. Calculation versioning
 4. Recalculation
-

Phase 9 — Evidence

1. Evidence model
 2. Decision Trace
 3. calculation basis UI
 4. policy clause viewer
-

Phase 10 — Review

1. Review queue
 2. expert review
 3. additional-document request
 4. override auditing
-

32. DO NOT

The following are prohibited unless the architecture is formally changed.

1. **DO NOT use an LLM response directly as the insurance benefit amount.**
2. **DO NOT use raw OCR values as authoritative calculation facts.**
3. **DO NOT guess the applicable policy version.**
4. **DO NOT silently use the latest policy.**
5. **DO NOT invent insurance payment rates.**
6. **DO NOT invent exclusions.**
7. **DO NOT invent waiting periods.**
8. **DO NOT invent disability rates.**
9. **DO NOT hardcode product-specific policy rules into generic application code.**

10. **DO NOT activate AI-extracted rules without human review and testing.**
 11. **DO NOT allow arbitrary code execution inside the rule DSL.**
 12. **DO NOT overwrite historical calculation records.**
 13. **DO NOT use binary floating point for authoritative money calculations.**
 14. **DO NOT return 0 KRW when a calculation actually failed.**
 15. **DO NOT expose private medical documents through public URLs.**
 16. **DO NOT skip object-level authorization.**
 17. **DO NOT store secrets in source code.**
 18. **DO NOT commit production credentials.**
 19. **DO NOT log full medical documents or authentication secrets.**
 20. **DO NOT confuse OCR confidence with payment probability.**
 21. **DO NOT mark unresolved insurance questions as certain.**
 22. **DO NOT silently remove Evidence when changing calculation logic.**
 23. **DO NOT modify an unrelated domain simply to avoid using the intended interface.**
 24. **DO NOT perform unnecessary large-scale refactors during a small feature task.**
 25. **DO NOT claim completion while required tests are failing.**
-

33. MVP Happy Path

The project must ultimately support this end-to-end scenario.

```
User logs in
↓
User registers insurance contract
↓
Contract contains:
Acute myocardial infarction diagnosis coverage
Insured amount 30,000,000 KRW
↓
User creates Claim
↓
User uploads diagnosis certificate
↓
```

```
Document validation succeeds
↓
OCR executes
↓
AI extracts:
Diagnosis = acute myocardial infarction
Code = I21.9
Diagnosis date = 2026-07-10
↓
User verifies extracted facts
↓
Facts become VerifiedFacts
↓
Policy Version Resolver resolves applicable PolicyVersion
↓
Coverage Matcher identifies subscribed AMI diagnosis coverage
↓
Eligibility Engine runs versioned rules
↓
Result = PAYABLE
↓
Calculation Engine receives:
VerifiedFact
ContractCoverage
PolicyVersion
RuleVersion
↓
Calculation Engine deterministically calculates:
30,000,000 KRW
↓
Evidence Service connects:
Calculation
→ RuleVersion
→ PolicyClause
→ VerifiedFact
→ diagnosis certificate page/location
↓
User receives:
Estimated insurance benefit
+
assessment status
+
calculation basis
+
policy evidence
```

The MVP is not successful merely because OCR or AI extraction works.

The MVP is successful when the complete chain works:

```
Document
→ Verified Fact
→ Policy
```

- Rule
- Assessment
- Calculation
- Evidence

34. Completion Report Format

When completing a development task, report in the following structure.

Implemented

- Describe the completed functionality.

Changed Files

- `path/to/file`
- `path/to/file`

Domain Impact

- Domain:
- Entities affected:

Database Changes

- None

or

- Migration:
- Tables/columns:
- Backward compatibility:

API Changes

- Added:
- Changed:
- Removed:

Policy / Rule Impact

- None

or

- Rule:
- Rule version:
- Regression impact:

Calculation Impact

- None

or

- Calculation behavior changed:
- Historical calculations preserved:
- Recalculation required:

Security / Privacy Impact

- Authorization:
- Sensitive data:
- Logging:
- External provider data:

Tests

- Unit:
- Integration:
- Rule regression:
- Security:
- E2E:

Validation Performed

- Build:
- Lint:
- Type check:
- Tests:
- Migration:

Remaining Issues

- None

or

- Clearly list unresolved items.

Final Project Invariant

The following invariant overrides implementation convenience:

```
Verified Facts
+
Contract Coverage
+
Policy Version
+
Rule Version
=
Coverage Assessment
+
Benefit Calculation
+
Evidence
```

Never let the LLM directly determine an insurance benefit amount.

Only verified facts, versioned policy rules, and deterministic calculation logic may produce an authoritative benefit calculation.

Every material benefit decision must be traceable to its contract, policy version, rule version, verified facts, calculation inputs, and source evidence.